1 BASICS (Wk01)

Variables & types

- Name: letters/digits/_, **not** start with digit, case-sensitive, no keywords, no `-` or space. Snake_case (convention only).
- Types: `int` `float` `bool` `str` `NoneType`. `type(x)` -> `<class 'int'>`
- `=` is **assignment** (old value replaced). Swap: `a, b = b, a`. Multi: `x, y = 1, 2`

```
int(23.8)      # 23 (truncates)
  float('45.2') # 45.2
str(123)       # '123'
  int('10 plus 20') # ValueError
int('3.5')     # ValueError
  bool(0)/bool('')/bool([]) # False
```

Operators (high -> low precedence)

Level	Operators
1	`**` (right-assoc) then unary `-`
2	`* / // %` (left to right)
3	`+ -`
4	`< <= > >= == != in`
5	`not` > `and` > `or`

```
11/3 -> 3.666  11//3 -> 3  11%3 -> 2  2**3
  -> 8
-7//2 -> -4 (floors!)  -7%2 -> 1   15/5
  -> 3.0 (/ is always float)
6/2*(1+2) -> 9.0     5%2==1 -> True
```

- `and`/`or` **short-circuit** & return an operand: `True and 0` -> `0`; `1 or 0` -> `1`; `False and (1/0)` -> `False` (no error); `True or 1/0` -> `True`.
- **Falsy**: `0, 0.0, "", [], (), {}, set(), None`. Everything else truthy. `True==1, False==0` so `not False + 1` = `not 1` = `False`.

TRAP: `==` vs `=` (#1 trap). `'1' == 1` -> False. `'a' + 3` -> TypeError. `'a'*3` fine. `print(a,b)` puts a space; `input()` always returns **str** -> `int(input())`.

Strings (immutable sequence of chars)

```
s = 'abcdef'    # idx:  0 1 2 3 4 5
  | negative: -1 = last
s[0] 'a'   s[-1] 'f'   s[6]
  # IndexError (single index)
s[start:stop:step]
  # stop EXCLUSIVE; defaults 0, len, 1
s[1:4] 'bcd'  s[:2] 'ab'  s[::2] 'ace'  s[:
  :-1] 'fedcba'
s[1:9] 'bcdef'  s[3:5] ''
  # slice out of range = NO error
'abc'[-5:-10] ''
  # start must be left of stop (step>0)
'ba' + 'na'*2  # 'banana'
  'z' in 'banana' # False
'bananb' > 'banana'
  # True (lexicographic, left to right)
s[0] = 'z'     # TypeError (immutable)
len('IT5001 Good') # 11 (spaces count)
f"{x}x tea at ${price}"
  # f-string: {} evaluates expr
```

Functions & modules

```
def square(x):
  # keyword, name, parameter, colon
    return x * x
      # indented body. no return -> None
square(3,4)              # TypeError (wrong
  # of args)
import math
  # math.cos(0), math.pi, math.sqrt
from math import cos, pi
  # use cos() directly
import random
  # random.randint(0,99)
    -> 0..99 BOTH incl.
```

- `print` shows a value (returns None); `return` hands it back. Code after `return` never runs.
- Function **abstraction**: name a block, reuse it. Indentation (4 spaces) defines the body; the first un-indented line is outside.
- `import math` avoids name clashes (need `math.`); `from math import x` shorter. `math.sqrt(-1)` -> ValueError (check `delta < 0` first).

2 SELECTION & LOOPS (Wk02)

```
if cond1:
  # only ONE branch runs: first True wins
    ...
elif cond2:
  # many elif allowed, else optional
    ...
else:
    ...
```

- Nested `if` must be indented; an `if` with no match and no `else` prints nothing.
- Order matters in `elif` chains: test most specific / use ranges low->high (`mark < 40`, `< 45`, `< 55` ...).

for / range / while

```
for x in <iterable>:
  # str, list, tuple, set, dict, range, fil
    e
    body
range(stop)  range(start,stop)  range(start
  ,stop,step)
range(5)->0..4  range(2,5)->2,3,4  range(5,
  0,-1)->5..1
range(0,5,-1) # EMPTY (wrong direction)
  range(1,n+1)->1..n
while cond:
  # repeat while True; must change cond!
    body
      # while True: ... = infinite
```

- **for**: count known beforehand. **while**: unknown # of repeats (depends on a condition changing inside).
- `break` = leave innermost loop now. `continue` = skip to next iteration (in `while`, update the counter **before** `continue`, else infinite loop).
- Loop patterns: **exactly N** (`for`); **check all** (return False at first bad, `return True` after loop); **find any** (return True at first hit, else False); **accumulate** (`total = 0` before loop).

```
def all_alpha(s):
  # "check all" pattern
    for c in s:
        if not c.isalpha(): return False
          # early exit
    return True
      # only after full loop
# digits of a number:  d = n % 10 ; n = n /
  / 10   (or: for d in str(n))
# nested loops: inner runs fully for each o
  uter value
```

TRAP: `while` loops: off-by-one (`<` vs `<=`). `range` stop excluded. `x //= 4` repeat until 0: count steps by hand.

3 SCOPE & CALL STACK (W03a)

- **Golden rule**: `x = ...` inside a function creates a **new local** x (global untouched). Params are locals.
- Reading only (no assignment in function) -> uses the **global** x. Lookup is in the *defining* file scope, **not** the caller's locals.

```
x = 0                  # case A: local copy
def f():
    x = 999; print(x)   # 999
f(); print(x)
  # 0  (global unchanged)
```

```
x = 0
  # case B: use before assign
def f():
    print(x)
      # UnboundLocalError!
    x = 999
      # makes x local for whole f
def g():                # case C: read only
  -> global
    print(x)            # 0 (no error)
```

```
def h():
  # case D: global keyword
    global x; x = 999
h(); print(x)           # 999
```

- Using a local before its assignment in the same function -> **UnboundLocalError**. `global x` lets you modify the global.
- Globals: use as UPPERCASE constants; pass data via params/return.
- Local vars die when function returns. Nested `def` is local to the outer function only.

```
def a(l): l = 'new'; print(l)
  # rebinding: outer l unchanged
l = 'old'; a(l); print(l)       # new / old
```

Call stack (FILO: first in, last out)

- Each call pushes a frame (own locals). When a function returns, its frame pops and control goes back to the caller. Output order: p1 in, p2 in, p3 in, p3 end, p2 end, p1 end.

4 RECURSION (W03a, W03c)

- Function calls itself on a **smaller** problem. Needs (1) **base case** (stop) (2) step that **shrinks** the input. Missing base -> `RecursionError` (~1000 deep).

```
def fact(n):
  # n! = n * (n-1)!, 0! = 1
    if n == 0: return 1            # base
    return n * fact(n-1)
      # fact(3)=3*2*1*1 = 6
def fib(n):
  # f(0)=f(1)=1, f(n)=f(n-1)+f(n-2)
    if n <= 1: return 1
      # 1,1,2,3,5,8,13...
    return fib(n-1) + fib(n-2)
      # 2 calls each -> exponential
def rev(s):
  # reverse string
    if s == '': return ''
    return rev(s[1:]) + s[0]
      # 'abc'->rev('bc')+'a'
def sumdig(n):                     # 52634
  -> 20
    if n == 0: return 0
    return n % 10 + sumdig(n // 10)
```

Iteration <-> Recursion conversion

Iteration	Recursion
initial value (`res = 1`)	base case return
loop body computation	kth term op answer(k-1)
next value (`n = n-1`)	argument of recursive call
stop (`while n > 0`)	base case condition
temp result variables	not needed (return value carries)

- Counting calls (e.g. `pascal(5,2)` prints): draw the call tree; each call that executes `print` first counts, including base-case calls.
- Iterative tuple/list build: `res = res + (x,)` / `res.append(x)`; recursive: `[x] + f(rest)`. Taylor/series: kth term + answer(k-1).
- **Refactoring** = restructure code without changing external behaviour.

5 SEQUENCES (Wk04a/b)

Type	Syntax	Mutable	Indexed	Notes
str	`'ab'`	No	Yes	chars
list	`[1,2]`	**Yes**	Yes	any types
tuple	`(1,2)`	No	Yes	`(1,)` singleton
set	`{1,2}`	Yes	**No**	unique, unordered
dict	`{k:v}`	Yes	by key	keys immutable

Works on all indexed sequences (str, list, tuple)

```
a[i]  a[i:j]  a[i:j:k]  len(a)  min(a)  max
  (a)  sum(a)
x in a   a + b (same type!)   n * a
  a[::-1]
for x in a:  (any iterable)    list('abc')
  -> ['a','b','c']
str([1,2]) -> '[1, 2]' (not '12')
  ''.join(['a','b']) -> 'ab'
```

List

```
L = [1, 'a', [2,3]]
  # mixed types, nesting OK
L.append(4)
  # add ONE item at end, returns None
L.append([5,6])   # nested: [..,[5,6]]   vs
  L += [5,6] / extend -> flat
L = L + [7]       # NEW list (rebinds)
  L += [7] modifies IN PLACE
L.remove(2)
  # removes FIRST 2 only; ValueError if abs
    ent
L.pop() / L.pop(i) / del L[i] / L.insert(i,
  x) / L.sort() / L.reverse()
L.sort()
  # in place, returns None  | sorted(L)
    -> new list
L = L.append(1)    # L is now None !!
L[1:] = ['x','y','z']
  # slice assign can change length
[0]*3 -> [0,0,0]   [[0]*3]*2
  # SAME inner list twice!
```

List comprehension [expr for x in iter if cond]

```
[i for i in range(1,101)]          # 1..100
[i**2 for i in range(1,11)]
  # first 10 squares
[i for i in range(1,100,2)]
  # odds < 100
[x for x in L if x % 2 == 0]
  # filter evens
[c for c in 'hello' if c in 'aeiou']
  # ['e','o']
non = [j for i in range(2,8) for j in range
  (2*i,50,i)]
primes = [n for n in range(2,50) if n not i
  n non]
```

Tuple

```
t = (1,2,3)  t[0]=9 # TypeError    (1)
  -> int    (1,) -> tuple   () empty
x, y = t
  # unpack (counts must match)
t + (4,)
  # new tuple; t += (4,) rebinds new tuple
tup = ([1,2,3], 4); tup[0][0] = 999
  # OK! inner list is mutable
tuple('IT') -> ('I','T')   t[1:]
  -> tuple (slice keeps type)
```

- Convention: **list** = many same-type items; **tuple** = small record of mixed items (breaking it is not an error). `[(x,y),..]` = list of tuples.

Aliasing / copying / pass-by-value

```
a = 5; b = a; b = 9
  # ints: a still 5 (independent copy)
A = [1,2]; B = A
  # ALIAS: same list object
B.append(3)  -> A is [1,2,3]  too
C = A[:] / A.copy() / list(A)
  # shallow copy: new outer list
N = [[1],[2]]; M = N[:]; M[0][0]=9
  # N changed too (shallow!)
la=[9,8,7]; lb=[la,la]; lb[0][0]=0
  # lb=[[0,8,7],[0,8,7]]
t1=[1]; t2=(t1,t1); t1.append(2)
  # t2=([1,2],[1,2])
```

- Function args: a copy of the **reference** is passed. `lst.append(x)` / `lst[0]=..` **modifies caller's list**. `lst = lst + [x]` rebinds local only.
- Tuple/int version must **return** the new value (original preserved). Tuple `t = t + (x,)` copies all items each time -> 1+2+..+n steps (slow); `list.append` is O(1) -> lists far faster for repeated adds.

Set {} (unordered, unique, no index)

```
s = {1,2,2,3} -> {1,2,3}    set([3,3,3])
  -> {3}   set('abc') chars
{}  # is an EMPTY DICT!
  empty set: set()
s[0] # TypeError
  len(s)  x in s  fine; + and * NOT allowed
s.add(4)   s.remove(9) # KeyError if absent
  s.discard(9) # no error
A|B union   A&B intersect   A-B in A not B
  A^B in exactly one
set('bananas') & set('pyjamas')
  -> {'a','s'}
{(1,3), 9}   # tuples OK;  {[1],2}
  # TypeError (list unhashable)
```

Dictionary {key: value} (fast lookup)

```
d = {'apples':450, 'oranges':200}
d['apples']        # 450       d['pears']
  # KeyError
d.get('pears')     # None (no crash)
  d.get('pears', 0) # 0
d['pears'] = 100   # add; if key exists
  -> OVERWRITES value
d.pop('apples')
  # removes (KeyError if absent)
    del d['oranges']
d.keys() d.values() d.items() d.clear() d.c
  opy()  # copy is shallow
'apples' in d
  # checks KEYS only (8 in {..:8} -> False)
for k in d: / for k,v in d.items():
  len(d)
dict([('A',2),(1,4)]) -> {'A':2, 1:4}
  keys: int/str/tuple OK
d[[8]] = 9
  # TypeError (list key unhashable)
play[play['rock']]
  # chained lookup: evaluate inside-out
```

- Keys unique & immutable; values anything. Dict lookup by key, not index. Swap/reverse lookup needs a loop over `items()`.

6 FILE I/O (Wk06a)

```
with open('f.txt', 'w') as f:
  # auto-closes; use f INSIDE block
    f.write('hi\n')
      # str only (str(5)); NO auto newline
with open('data.csv') as f:
  # default mode 'r' (text)
    f.readline()
      # next line incl '\n' ('' at EOF)
    for line in f:
      # file is ITERABLE: line by line
        row = line.rstrip('\n').split(',')
          # ['1960','x','37.5']
        yr  = int(row[0]); v = float(row[2]
          )  # convert from str!
    all = f.read()
      # whole file as ONE str (bad if huge)
```

Mode	Meaning
`'r'`	read (default); missing file -> FileNotFoundError
`'w'`	write; creates / **erases** existing
`'a'`	append at end
`'b'`	binary (add to mode, e.g. 'rb')

- `split()` no arg = split on spaces/newlines; `split(',')` on commas; `rstrip('\n')` removes trailing newline. 1st header line: call `f.readline()` once to skip.
- Reading line gives str -> convert with `int()`/`float()`. `csv.reader(f)` is the easy way (`import csv`). Plot: `plt.plot(x,y,label=..)`, `plt.legend()`, `plt.title()`, `plt.show()`.

TRAP: Using `f` after the `with` block ends -> ValueError (closed). Hard-coding data in code is bad; read from a file.

7 HIGHER-ORDER FUNCTIONS (Wk06b)

- Functions are **values**: can be stored, passed, returned, put in lists. `f` = the function; `f(x)` = call it (result).

```
x = square
  # x is now the same function
x(3) -> 9      type(square)
  -> <class 'function'>
fs = [abs, len]; fs[1]([1,2,3]) # 3
  compose: log(sin(x))
def dist(a,b):
    def sq(z): return z*z
      # local function: invisible outside
    return (sq(a)+sq(b))**0.5
```

lambda (anonymous function)

```
lambda x: x**2
  # no 'return', ONE expression
(lambda x: x**2)(5)        # 25
  call immediately
sq = lambda x, y: x*x + y*y   # same as def
def make_pow(n): return lambda x: x**n
  # returns a function
make_pow(3)(2) -> 8
  # cube = make_pow(3); cube(2)
def make_div(n): return lambda x: x % n ==
  0
(lambda p: lambda q: p(p(q)))(lambda x: x%1
  0)(1432) # 2
#   substitute step by step: apply p twice
  to q
lambda P,r,n: P*(1 + r/100)**n
  # watch brackets!
```

Functions that take / return functions

```
def deriv(f, dx=0.0001):
  # takes f, returns f'
    return lambda x: (f(x+dx) - f(x)) / dx
def newton(g, x=1.0):
  # root of g: x - g(x)/g'(x)
    dg = deriv(g)
    while abs(g(x)) > 1e-9: x = x - g(x)/dg
      (x)
    return x
newton(lambda x: x*x - 2)
  # sqrt(2) ; log10(A): 10**x - A
```

8 MAP, FILTER, DEEP OPS (Wk07)

```
def my_map(f, L):
  # own version (lists only)
    if L == []: return []
    return [f(L[0])] + my_map(f, L[1:])
m = map(lambda x: x*2, [1,2,3])
  # map OBJECT (lazy iterable)
list(m) -> [2,4,6]    list(m) -> []
  # CONSUMED after one use!
tuple(map(str, [1,2]))
  list(map(f, 'abc'))   # any sequence
filter(pred, seq)
  # keep x where pred(x) is True (pred
    -> bool)
list(filter(lambda x: x%2==0, range(6)))
  -> [0,2,4]
```

Shallow vs deep (nested lists)

```
def deepcount(s):
  # leaves count 1
    if s == []: return 0
    if type(s) != list: return 1
      # or not isinstance(s,list)
    return deepcount(s[0]) + deepcount(s[1:
      ])
deepcount([[[1,2],3],9,[1,2]]) -> 6
  len(..) -> 3 (shallow)
def deepmap(f, s):
  # deepsq: f = x*x
    if s == []: return []
    if type(s) != list: return f(s)
      # leaf: apply f
    return [deepmap(f, s[0])] + deepmap(f,
      s[1:])  # [ ] keeps nesting
def flatten(s):
    if s == []: return []
    if type(s) != list: return [s]
    return flatten(s[0]) + flatten(s[1:])
      # flatten([[[1]],2])->[1,2]
```

- Pattern: (1) empty base (2) **leaf** case (3) first item recursed + rest recursed. Wrap first result in `[ ]` to keep structure; omit to flatten.
- `copy()`/`[:]` = **shallow** (inner lists shared); `deepcopy` = `deepmap(lambda x:x, s)` copies every level. Real uses: directories, trees, images (rows->cols->RGB).
- Check type: `type(x) == list`, `isinstance(x, list)`; tuples: `type(x) in (list, tuple)`.

9 EXAM CHECKLIST (from past papers)

- Paper: ~25 MCQ (3 marks) + 5 fill-in-blank (5 marks), 1 hour. Mostly **predict the output**; FITB = loops, `if` ranges, recursion, lambda return.

Topic asked	Check this
Expressions	precedence, `/` float, `//` floor, short-circuit, falsy
Strings	slice out of range, negative idx, step, `[-1::-1]`
Lists	alias vs `[:]`, `sort()` in place, `+` vs append
Tuples	`(1,)`, mutable inside, `+=` builds new, `range(0,5,-1)`
Sets/dicts	dups removed, `^ & - |`, `.get` None, key unhashable
Loops	trace table; `break`/`continue`; digit loops `%10`, `//10`
Scope	which `x`? local vs global; param reassign
Lambda	expand inside-out; `make_function(n)` returns lambda
Recursion	count calls; `n%2==0 and f(n//2)`; deep_convert

TRAP: Always: write var values line by line; "error" is an option - look for TypeError / KeyError / IndexError / NameError / ValueError.